

2303A51833

S.Aishwarya - 26

Assignment Number:11.3

Q.No.	Question	<i>Expected Time to complete</i>
1	Lab 11: Data Structures with AI Implementing Fundamental Data Structures using AI Assistance Lab Objectives: By the end of this lab, students will be able to: • Design and implement fundamental data structures in Python using AI assistance. • Effectively prompt AI tools (e.g., GitHub Copilot) for code generation, optimization, and documentation. • Understand and compare core data structures: Arrays, Linked Lists, Stacks, Queues, Priority Queues, Trees, and Graphs. • Improve code readability, efficiency, and maintainability using AI-generated suggestions. Learning Outcomes After completing this lab, students will be able to: • Apply appropriate data structures to solve real-world problems. • Analyze time and space complexity of different data structure operations. • Use AI tools responsibly to assist (not replace) logical thinking and problem-solving. • Validate, test, and refine AI-generated code. Task 1: Smart Contact Manager (Arrays & Linked Lists) Scenario SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts. Tasks 1. Implement the contact manager using arrays (lists). 2. Implement the same functionality using a linked list for dynamic memory allocation. 3. Implement the following operations in both approaches: ○ Add a contact ○ Search for a contact ○ Delete a contact 4. Use GitHub Copilot to assist in generating search and delete methods. 5. Compare array vs. linked list approaches with respect to:	Week6 - Wednesday

- Insertion efficiency
- Deletion efficiency

Expected Outcome

- Two working implementations (array-based and linked-list-based).
- A brief comparison explaining performance differences.

Prompt :

Implement a Smart Contact Manager for SR University's student club using two approaches: arrays (Python lists) and linked lists. Both versions should support adding, searching, and deleting contacts (name + phone number). Use GitHub Copilot to assist in generating search and delete methods. Finally, compare array vs. linked list approaches in terms of insertion and deletion efficiency, highlighting trade-offs in performance and memory usage.

Code :

```
Reasoning: The subtask requires defining a Python class named Contact with specific attributes and methods, so I will provide the Python code for this class.

(1) ✓ Os class Contact:
    def __init__(self, name, phone_number):
        self.name = name
        self.phone_number = phone_number

    def __str__(self):
        return f'Name: {self.name}, Phone: {self.phone_number}'

    print("Contact class defined successfully.")

--- Contact class defined successfully.

Implement Array-based Contact Manager

Subtask:
Develop a contact manager class using a Python list (acting as an array) to store contact objects. This implementation will include methods for adding new contacts, searching for existing contacts (with GitHub Copilot's assistance for the method), and deleting contacts (also with Copilot's assistance).

class ArrayContactManager:
    def __init__(self):
        self.contacts = []
        print("ArrayContactManager initialized with an empty list.")

    def add_contact(self, contact):
        self.contacts.append(contact)
        print(f"Added contact: {contact.name}")

    def search_contact(self, name):
        for contact in self.contacts:
            if contact.name == name:
                return contact
        return None

    def delete_contact(self, name):
        original_len = len(self.contacts)
        self.contacts = [contact for contact in self.contacts if contact.name != name]
        if len(self.contacts) < original_len:
            print(f"Deleted contact: {name}")
            return True
        print(f"Contact not found for deletion: {name}")
        return False

    def __str__(self):
        if not self.contacts:
```

```

    def __str__(self):
        if not self.contacts:
            return "No contacts in the manager."
        return "\n".join(str(contact) for contact in self.contacts)

    print("ArrayContactManager class defined successfully.")

ArrayContactManager class defined successfully.

```

Reasoning: Now that the `ArrayContactManager` class is defined, I will demonstrate its functionality by creating an instance, adding contacts, searching for contacts, and deleting contacts. This will verify the implementation of the class methods.

```

1) 0s ▶ print("\n--- Testing ArrayContactManager ---")
    manager_array = ArrayContactManager()

    # Create some contacts
    contact1 = Contact("Alice", "111-222-3333")
    contact2 = Contact("Bob", "444-555-6666")
    contact3 = Contact("Charlie", "777-888-9999")

    # Add contacts
    manager_array.add_contact(contact1)
    manager_array.add_contact(contact2)
    manager_array.add_contact(contact3)

0) 0s ▶ print("\nCurrent contacts in manager:")
    print(manager_array)

    # Search for a contact
    print("\nSearching for Bob:")
    found_contact = manager_array.search_contact("Bob")
    if found_contact:
        print(f"Found: {found_contact}")
    else:
        print("Bob not found.")

    print("\nSearching for David (non-existent):")
    found_contact = manager_array.search_contact("David")
    if found_contact:
        print(f"Found: {found_contact}")
    else:
        print("David not found.")

    # Delete a contact
    print("\nDeleting Bob:")
    manager_array.delete_contact("Bob")

    print("\nContacts after deleting Bob:")
    print(manager_array)

    print("\nAttempting to delete non-existent contact (Frank):")
    manager_array.delete_contact("Frank")

▶ print("\nFinal contacts in manager:")
  print(manager_array)

  print("--- ArrayContactManager testing complete ---")

```

```

...
--- Testing ArrayContactManager ---
ArrayContactManager initialized with an empty list.
Added contact: Alice
Added contact: Bob
Added contact: Charlie

Current contacts in manager:
Name: Alice, Phone: 111-222-3333
Name: Bob, Phone: 444-555-6666
Name: Charlie, Phone: 777-888-9999

Searching for Bob:
Found: Name: Bob, Phone: 444-555-6666

Searching for David (non-existent):
David not found.

Deleting Bob:
Deleted contact: Bob

Contacts after deleting Bob:
Name: Alice, Phone: 111-222-3333
Name: Charlie, Phone: 777-888-9999

```

```
[4]
✓ Os
class Node:
    def __init__(self, data):
        self.data = data # Stores a Contact object
        self.next = None # Reference to the next node

    print("Node class defined successfully.")

Node class defined successfully.
```

Reasoning: Now that the `Node` class is defined, I will define the `LinkedListContactManager` class as specified, including its `__init__`, `add_contact`, `search_contact`, `delete_contact`, and `__str__` methods to manage contacts using a linked li

```
[5]
✓ Os
class LinkedListContactManager:
    def __init__(self):
        self.head = None
        print("LinkedListContactManager initialized with an empty linked list.")

    def add_contact(self, contact):
        new_node = Node(contact)
        if self.head is None:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
```

```

            current = current.next
            current.next = new_node
            print(f"Added contact to linked list: {contact.name}")

    def search_contact(self, name):
        current = self.head
        while current:
            if current.data.name == name:
                return current.data
            current = current.next
        return None

    def delete_contact(self, name):
        current = self.head
        prev = None

        # Case 1: Contact to be deleted is the head
        if current and current.data.name == name:
            self.head = current.next
            print(f"Deleted contact from linked list: {name}")
            return True

        # Case 2: Search for the contact to be deleted
        while current and current.data.name != name:
            prev = current
            current = current.next

        # If contact not found
```

```

        # Unlink the node from the linked list
        if prev:
            prev.next = current.next
        else: # This case should ideally be handled by Case 1, but for robustness
            self.head = current.next

        print(f"Deleted contact from linked list: {name}")
        return True

    def __str__(self):
        if self.head is None:
            return "No contacts in the linked list manager."

        contacts_str = []
        current = self.head
        while current:
            contacts_str.append(str(current.data))
            current = current.next
        return "\n".join(contacts_str)

    print("LinkedListContactManager class defined successfully.")
```

```
... LinkedListContactManager class defined successfully.
```

```

▶ print("\n--- Testing LinkedListContactManager ---")
manager_linked_list = LinkedListContactManager()

# Create some contacts (reusing the Contact class from earlier)
contact4 = Contact("David", "111-222-1111")
contact5 = Contact("Eve", "222-333-2222")
contact6 = Contact("Frank", "333-444-3333")

# Add contacts
manager_linked_list.add_contact(contact4)
manager_linked_list.add_contact(contact5)
manager_linked_list.add_contact(contact6)

print("\nCurrent contacts in linked list manager:")
print(manager_linked_list)

# Search for a contact
print("\nSearching for Eve:")
found_contact = manager_linked_list.search_contact("Eve")
if found_contact:
    print(f"Found: {found_contact}")
else:
    print("Eve not found.")

print("\nSearching for Grace (non-existent):")
found_contact = manager_linked_list.search_contact("Grace")
if found_contact:

```

```

--- Testing LinkedListContactManager ---
LinkedListContactManager initialized with an empty linked list.
...
Added contact to linked list: David
Added contact to linked list: Eve
Added contact to linked list: Frank

Current contacts in linked list manager:
Name: David, Phone: 111-222-1111
Name: Eve, Phone: 222-333-2222
Name: Frank, Phone: 333-444-3333

Searching for Eve:
Found: Name: Eve, Phone: 222-333-2222

Searching for Grace (non-existent):
Grace not found.

Deleting Eve:
Deleted contact from linked list: Eve

Contacts after deleting Eve:
Name: David, Phone: 111-222-1111
Name: Frank, Phone: 333-444-3333

Attempting to delete non-existent contact (Henry):
Contact not found for deletion in linked list: Henry

Final contacts in linked list manager:
Name: David, Phone: 111-222-1111
Name: Frank, Phone: 333-444-3333
--- LinkedListContactManager testing complete ---

```

Explanation : The comparative analysis reveals that for a contact manager where contacts are added to the end, searched by name, and deleted by name, the array-based approach (using Python's list) is generally more practical and performant. Its strengths include amortized $O(1)$ time complexity for adding contacts and lower memory overhead per contact due to Python's optimized list implementation. The linked list-based approach, in its current implementation, has a weakness of $O(N)$ for adding contacts (due to traversal for appending) and higher memory overhead per contact because each contact requires an additional Node object. Both approaches share a weakness of $O(N)$ time complexity for searching and deleting by name, as they rely on linear traversal.

Task 2: Library Book Search System (Queues & Priority Queues)

Scenario

The SRU Library manages book borrow requests. Students and faculty submit requests, but **faculty requests must be prioritized** over student requests.

Tasks

1. Implement a **Queue (FIFO)** to manage book requests.

2. Extend the system to a **Priority Queue**, prioritizing faculty requests.
3. Use **GitHub Copilot** to assist in generating:
 - enqueue() method
 - dequeue() method
4. Test the system with a mix of student and faculty requests.

Expected Outcome

- Working queue and priority queue implementations.
- Correct prioritization of faculty requests.

Prompt :

Develop a Library Book Search System for SRU Library using queues and priority queues. First, implement a standard FIFO queue to manage book borrow requests. Then extend it to a priority queue where faculty requests are served before student requests. Use GitHub Copilot to assist in generating the enqueue() and dequeue() methods. Finally, test the system with a mix of student and faculty requests to ensure correct prioritization.

Code :

```
[?]
0s class BookRequest:
    def __init__(self, book_title, requestor_name, requestor_type):
        self.book_title = book_title
        self.requestor_name = requestor_name
        self.requestor_type = requestor_type

    def __str__(self):
        return f"Book: {self.book_title}, Requestor: {self.requestor_name}, Type: {self.requestor_type}"

    print("BookRequest class defined successfully.")

BookRequest class defined successfully.

1
0s class FIFOQueue:
    def __init__(self):
        self.queue = []
        print("FIFOQueue initialized.")

    def enqueue(self, request):
        self.queue.append(request)
        print(f"Enqueued request: {request.book_title} by {request.requestor_name}")

    def dequeue(self):
        if self.is_empty():
            print("Queue is empty. No requests to dequeue.")
            return None
        request = self.queue.pop(0)
        print(f"Dequeued request: {request.book_title} by {request.requestor_name}")
        return request

    def is_empty(self):
        return len(self.queue) == 0

    def __len__(self):
        return len(self.queue)

    def __str__(self):
        if self.is_empty():
            return "FIFO Queue: Empty"
        requests_str = "\n".join(str(req) for req in self.queue)
```

```
[9] print("FIFOQueue class defined successfully.")
✓ Os
FIFOQueue class defined successfully.
```

Reasoning: Now that the `FIFOQueue` class is defined, I will demonstrate its functionality by creating an instance, enqueueing several `BookRequest` objects, checking the queue's state, dequeuing requests, and verifying the `is_empty` and `__str__` methods. This will confirm that the FIFO queue behaves as expected.

```
[9] print("\n--- Testing FIFOQueue ---")
✓ Os
fifi_queue = FIFOQueue()

# Create some book requests
request1 = BookRequest("The Great Gatsby", "Alice", "student")
request2 = BookRequest("Calculus for Engineers", "Bob", "faculty")
request3 = BookRequest("Python Programming", "Charlie", "student")

# Enqueue requests
fifi_queue.enqueue(request1)
fifi_queue.enqueue(request2)
fifi_queue.enqueue(request3)

print("\nCurrent queue state:")
print(fifi_queue)

print("\nDequeuing first request:")
processed_request = fifi_queue.dequeue()
```

```
[9] print("\nCurrent queue state:")
✓ Os
print(fifi_queue)

print("\nDequeuing first request:")
processed_request = fifi_queue.dequeue()
if processed_request:
    print(f"Processed: {processed_request}")

print("\nCurrent queue state:")
print(fifi_queue)

print("\nDequeuing second request:")
processed_request = fifi_queue.dequeue()
if processed_request:
    print(f"Processed: {processed_request}")

print("\nCurrent queue state:")
print(fifi_queue)

print("\nDequeuing third request:")
processed_request = fifi_queue.dequeue()
if processed_request:
    print(f"Processed: {processed_request}")

print("\nCurrent queue state:")
print(fifi_queue)

print("\nAttempting to dequeue from empty queue:")
```

```
print("--- FIFOQueue testing complete ---")
```

```
...
--- Testing FIFOQueue ---
FIFOQueue initialized.
Enqueued request: The Great Gatsby by Alice
Enqueued request: Calculus for Engineers by Bob
Enqueued request: Python Programming by Charlie

Current queue state:
FIFO Queue (Current Requests):
Book: The Great Gatsby, Requestor: Alice, Type: student
Book: Calculus for Engineers, Requestor: Bob, Type: faculty
Book: Python Programming, Requestor: Charlie, Type: student

Dequeuing first request:
Dequeued request: The Great Gatsby by Alice
Processed: Book: The Great Gatsby, Requestor: Alice, Type: student

Current queue state:
FIFO Queue (Current Requests):
Book: Calculus for Engineers, Requestor: Bob, Type: faculty
Book: Python Programming, Requestor: Charlie, Type: student

Dequeuing second request:
Dequeued request: Calculus for Engineers by Bob
Processed: Book: Calculus for Engineers, Requestor: Bob, Type: faculty

Current queue state:
FIFO Queue (Current Requests):
```


Current queue state:
FIFO Queue (Current Requests):
Book: Python Programming, Requestor: Charlie, Type: student

Dequeuing third request:
Dequeued request: Python Programming by Charlie
Processed: Book: Python Programming, Requestor: Charlie, Type: student

Current queue state:
FIFO Queue: Empty

Attempting to dequeue from empty queue:
Queue is empty. No requests to dequeue.
--- FIFOQueue testing complete ---

```
class PriorityQueue:
    def __init__(self):
        self.queue = []
        print("PriorityQueue initialized.")

    def enqueue(self, request):
        # Prioritize faculty (priority 0) over student (priority 1)
        priority = 0 if request.requestor_type == "faculty" else 1

        # Insert the request while maintaining priority order
        # Faculty requests should appear before student requests
        # For requests of the same priority, maintain arrival order (FIFO)
        inserted = False
        for i, (existing_priority, existing_request) in enumerate(self.queue):
            if priority < existing_priority: # New request has higher priority
                self.queue.insert(i, (priority, request))
                inserted = True
                break
            # If same priority, do nothing special, it will be appended later
            # or inserted if it's faculty and existing is student

        if not inserted:
            self.queue.append((priority, request))

        print(f"Enqueued priority request: {request.book_title} by {request.requestor_name} (Priority: {request.requestor_type})")

    def dequeue(self):
        if self.is_empty():
```

Tooale Gemini

```
        print("Priority queue is empty. No requests to dequeue.")
        return None

    # The queue is maintained such that the highest priority element is always at the front,
    # or, if we have to scan, we'd pick the one with smallest priority number (0 for faculty).
    # Since we insert based on priority, the first element should be the highest priority.
    # If there are multiple items with the same highest priority, it maintains FIFO order.

    # Find the highest priority item (lowest priority number)
    highest_priority_item = None
    highest_priority_index = -1
    min_priority_value = float('inf')

    for i, (priority_value, request) in enumerate(self.queue):
        if priority_value < min_priority_value:
            min_priority_value = priority_value
            highest_priority_item = (priority_value, request)
            highest_priority_index = i

    if highest_priority_item:
        priority_value, request = self.queue.pop(highest_priority_index)
        print(f"Dequeued priority request: {request.book_title} by {request.requestor_name} (Priority: {request.requestor_type})")
        return request
    return None

    def is_empty(self):
        return len(self.queue) == 0

    def __len__(self):
        return len(self.queue)

    def __str__(self):
        if self.is_empty():
            return "Priority Queue: Empty"

        requests_str = []
        for priority, req in self.queue:
            requests_str.append(f"[P{priority}] {req}")
        return f"Priority Queue (Current Requests):\n" + "\n".join(requests_str)

print("PriorityQueue class defined successfully.")
```

PriorityQueue class defined successfully.


```

▶ print("\n--- Testing PriorityQueue ---")
priority_queue = PriorityQueue()

# Create some book requests with mixed priorities
request_s1 = BookRequest("History of Art", "Sarah", "student")
request_f1 = BookRequest("Advanced Physics", "Dr. Smith", "faculty")
request_s2 = BookRequest("Introduction to Philosophy", "Tom", "student")
request_f2 = BookRequest("Quantum Mechanics", "Prof. Johnson", "faculty")
request_s3 = BookRequest("Data Structures", "Umar", "student")

# Enqueue requests in a mixed order
priority_queue.enqueue(request_s1)
priority_queue.enqueue(request_f1)
priority_queue.enqueue(request_s2)
priority_queue.enqueue(request_f2)
priority_queue.enqueue(request_s3)

print("\nCurrent priority queue state:")
print(priority_queue)

print("\nDequeuing requests to observe priority:")
processed_request = priority_queue.dequeue() # Should be faculty (Dr. Smith)
if processed_request:
    print(f"Processed: {processed_request}")

processed_request = priority_queue.dequeue() # Should be faculty (Prof. Johnson)
if processed_request:
    print(f"Processed: {processed_request}")

```

```

▶ if processed_request:
    print(f"Processed: {processed_request}")

print("\nCurrent priority queue state after processing faculty:")
print(priority_queue)

processed_request = priority_queue.dequeue() # Should be student (Sarah)
if processed_request:
    print(f"Processed: {processed_request}")

processed_request = priority_queue.dequeue() # Should be student (Tom)
if processed_request:
    print(f"Processed: {processed_request}")

print("\nCurrent priority queue state:")
print(priority_queue)

processed_request = priority_queue.dequeue() # Should be student (Umar)
if processed_request:
    print(f"Processed: {processed_request}")

print("\nFinal priority queue state:")
print(priority_queue)

print("\nAttempting to dequeue from empty priority queue:")
priority_queue.dequeue()

print("--- PriorityQueue testing complete ---")

```

```

--- Testing PriorityQueue ---
PriorityQueue initialized.
Enqueued priority request: History of Art by Sarah (Priority: student)
Enqueued priority request: Advanced Physics by Dr. Smith (Priority: faculty)
Enqueued priority request: Introduction to Philosophy by Tom (Priority: student)
Enqueued priority request: Quantum Mechanics by Prof. Johnson (Priority: faculty)
Enqueued priority request: Data Structures by Umar (Priority: student)

Current priority queue state:
Priority Queue (Current Requests):
[P0] Book: Advanced Physics, Requestor: Dr. Smith, Type: faculty
[P0] Book: Quantum Mechanics, Requestor: Prof. Johnson, Type: faculty
[P1] Book: History of Art, Requestor: Sarah, Type: student
[P1] Book: Introduction to Philosophy, Requestor: Tom, Type: student
[P1] Book: Data Structures, Requestor: Umar, Type: student

Dequeuing requests to observe priority:
Dequeued priority request: Advanced Physics by Dr. Smith (Priority: faculty)
Processed: Book: Advanced Physics, Requestor: Dr. Smith, Type: faculty
Dequeued priority request: Quantum Mechanics by Prof. Johnson (Priority: faculty)
Processed: Book: Quantum Mechanics, Requestor: Prof. Johnson, Type: faculty

Current priority queue state after processing faculty:
Priority Queue (Current Requests):
[P1] Book: History of Art, Requestor: Sarah, Type: student
[P1] Book: Introduction to Philosophy, Requestor: Tom, Type: student
[P1] Book: Data Structures, Requestor: Umar, Type: student
Dequeued priority request: History of Art by Sarah (Priority: student)
Processed: Book: History of Art, Requestor: Sarah, Type: student
Dequeued priority request: Introduction to Philosophy by Tom (Priority: student)
Processed: Book: Introduction to Philosophy, Requestor: Tom, Type: student

Current priority queue state:
Priority Queue (Current Requests):
[P1] Book: Data Structures, Requestor: Umar, Type: student
Dequeued priority request: Data Structures by Umar (Priority: student)
Processed: Book: Data Structures, Requestor: Umar, Type: student

Final priority queue state:
Priority Queue: Empty

Attempting to dequeue from empty priority queue:
Priority queue is empty. No requests to dequeue.
--- PriorityQueue testing complete ---

```

Explanation :

For practical implementation at SRU Library, the Priority Queue is conceptually better because it fulfills the essential prioritization need. However, to achieve better performance for larger volumes of requests, the PriorityQueue implementation should be optimized. A more efficient implementation would typically use a min-heap (e.g., Python's `heapq` module) to manage the requests. A heap-based priority queue would offer $O(\log N)$ time complexity for both enqueue and dequeue operations, making it much more scalable than the current $O(N)$ list-based approach

Task 3: Emergency Help Desk (Stack Implementation)

Scenario

SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a **Last-In, First-Out (LIFO)** approach.

Tasks

1. Implement a **Stack** to manage support tickets.
2. Provide the following operations:
 - `push(ticket)`
 - `pop()`
 - `peek()`

3. Simulate at least **five tickets** being raised and resolved.
4. Use **GitHub Copilot** to suggest additional stack operations such as:
 - Checking whether the stack is empty
 - Checking whether the stack is full (if applicable)

Expected Outcome

- Functional stack-based ticket management system.
- Clear demonstration of LIFO behavior.

Prompt :

Build an Emergency Help Desk ticket management system for SR University using a stack (LIFO). Implement operations to push(ticket), pop(), and peek() tickets. Simulate at least five tickets being raised and resolved to demonstrate LIFO behavior. Use GitHub Copilot to suggest additional stack operations such as checking if the stack is empty or full (if applicable).

Code :

```
[15] print("\n--- Testing TicketStack ---")
✓ 0s ticket_manager = TicketStack(capacity=3)

# Create some tickets
ticket1 = Ticket("T001", "Printer not working", "High")
ticket2 = Ticket("T002", "Software installation request", "Medium")
ticket3 = Ticket("T003", "Network connectivity issue", "Critical")
ticket4 = Ticket("T004", "Monitor flickering", "High")

# Push tickets onto the stack
ticket_manager.push(ticket1)
ticket_manager.push(ticket2)
ticket_manager.push(ticket3)

print("\nCurrent stack state:")
print(ticket_manager)

print("\nAttempting to push when full:")
ticket_manager.push(ticket4)

print("\nPeeking at the top ticket:")
top_ticket = ticket_manager.peek()
if top_ticket:
    print(f"Top ticket: {top_ticket}")

print("\nPopping tickets:")
processed_ticket = ticket_manager.pop() # Should be T003
if processed_ticket:
```

```

    print(f"Processed: {processed_ticket}")

    print("\nCurrent stack state:")
    print(ticket_manager)

    processed_ticket = ticket_manager.pop() # Should be T002
    if processed_ticket:
        print(f"Processed: {processed_ticket}")

    print("\nCurrent stack state:")
    print(ticket_manager)

    processed_ticket = ticket_manager.pop() # Should be T001
    if processed_ticket:
        print(f"Processed: {processed_ticket}")

    print("\nFinal stack state (should be empty):")
    print(ticket_manager)
    print(f"Is stack empty? {ticket_manager.is_empty()}")

    print("\nAttempting to pop from empty stack:")
    ticket_manager.pop()

    print("--- TicketStack testing complete ---")

...
--- Testing TicketStack ---
TicketStack initialized with capacity: 3.
Pushed ticket: T001
Pushed ticket: T002
... Pushed ticket: T003

Current stack state:
Ticket Stack (Top to Bottom):
Ticket ID: T003, Description: 'Network connectivity issue', Priority: Critical
Ticket ID: T002, Description: 'Software installation request', Priority: Medium
Ticket ID: T001, Description: 'Printer not working', Priority: High

Attempting to push when full:
Stack is full. Cannot push more tickets.

Peeking at the top ticket:
Peeked ticket: T003
Top ticket: Ticket ID: T003, Description: 'Network connectivity issue', Priority: Critical

Popping tickets:
Popped ticket: T003
Processed: Ticket ID: T003, Description: 'Network connectivity issue', Priority: Critical

Current stack state:
Ticket Stack (Top to Bottom):
Ticket ID: T002, Description: 'Software installation request', Priority: Medium
Ticket ID: T001, Description: 'Printer not working', Priority: High
Popped ticket: T002
Processed: Ticket ID: T002, Description: 'Software installation request', Priority: Medium

Current stack state:
Ticket Stack (Top to Bottom):
Ticket ID: T001, Description: 'Printer not working', Priority: High
Popped ticket: T001
Processed: Ticket ID: T001, Description: 'Printer not working', Priority: High

Final stack state (should be empty):
Ticket Stack: Empty
Is stack empty? True

Attempting to pop from empty stack:
Stack is empty. No tickets to pop.
--- TicketStack testing complete ---

```

- Explanation : FIFO Queue Effectiveness: The standard FIFO queue was simple to implement and maintained the order of requests based on their arrival time. However, it did not provide any prioritization mechanism, meaning faculty requests arriving after student requests would be processed later. The dequeue operation for this implementation had a time complexity of $O(N)$ due to `list.pop(0)`, which involves shifting all subsequent elements.

Task 4: Hash Table

Objective

To implement a **Hash Table** and understand collision handling.

Task Description

Use AI to generate a hash table with:

- Insert
- Search
- Delete

Starter Code

```
class HashTable:
```

```
pass
```

Expected Outcome

- Collision handling using chaining
- Well-commented methods

Prompt :

Implement a Hash Table in Python to support insert, search, and delete operations. Use chaining (linked lists) to handle collisions. Begin with the starter class `class HashTable: pass` and expand it into a fully functional, well-commented implementation. Ensure each method demonstrates proper collision handling and clarity in design.

Code :

```
class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [[] for _ in range(self.size)]
        print(f"HashTable initialized with size: {self.size}.")

    def _hash_function(self, key):
        return hash(key) % self.size

    print("HashTable class defined successfully with __init__ and _hash_function.")

... HashTable class defined successfully with __init__ and _hash_function.

class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [[] for _ in range(self.size)]
        print(f"HashTable initialized with size: {self.size}.")

    def _hash_function(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self._hash_function(key)
        bucket = self.table[index]

        # Check if key already exists and update value
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
                print(f"Updated key '{key}' at index {index}.")
                return

        # If key does not exist, add the new key-value pair
        bucket.append((key, value))
        print(f"Inserted key '{key}' with value '{value}' at index {index}.")

    print("HashTable class defined successfully with __init__, _hash_function, and insert methods.")

... HashTable class defined successfully with __init__, _hash_function, and insert methods.
```

```

8] 1 print("\n--- Testing HashTable insert method ---")
Os hash_table = HashTable(size=5)

# Insert new key-value pairs
hash_table.insert("apple", 10)
hash_table.insert("banana", 20)
hash_table.insert("cherry", 30)

# Insert another key that might cause a collision (depending on hash function and size)
hash_table.insert("date", 40) # Likely different index
hash_table.insert("grape", 50) # May collide with apple if hash("apple") % 5 == hash("grape") % 5

# Update an existing key
hash_table.insert("apple", 15)

print("\n--- Current state of the HashTable buckets ---")
for i, bucket in enumerate(hash_table.table):
    print(f"Bucket {i}: {bucket}")

print("--- HashTable insert testing complete ---")

```

```

...
--- Testing HashTable insert method ---
HashTable initialized with size: 5.
Inserted key 'apple' with value '10' at index 4.
Inserted key 'banana' with value '20' at index 1.
Inserted key 'cherry' with value '30' at index 0.
Inserted key 'date' with value '40' at index 2.

Inserted key 'date' with value '40' at index 2.
Inserted key 'grape' with value '50' at index 3.
... Updated key 'apple' at index 4.

--- Current state of the HashTable buckets ---
Bucket 0: [('cherry', 30)]
Bucket 1: [('banana', 20)]
Bucket 2: [('date', 40)]
Bucket 3: [('grape', 50)]
Bucket 4: [('apple', 15)]
--- HashTable insert testing complete ---

```

```

[19] 1 class HashTable:
Os 2     def __init__(self, size):
3         self.size = size
4         self.table = [[] for _ in range(self.size)]
5         print(f"HashTable initialized with size: {self.size}.")

6     def _hash_function(self, key):
7         return hash(key) % self.size

8     def insert(self, key, value):
9         index = self._hash_function(key)
10        bucket = self.table[index]

11        # Check if key already exists and update value
12        for i, (k, v) in enumerate(bucket):
13            if k == key:
14                bucket[i] = (key, value)
15                print(f"Updated key '{key}' at index {index}.")
16                return

17        # If key does not exist, add the new key-value pair
18        bucket.append((key, value))
19        print(f"Inserted key '{key}' with value '{value}' at index {index}.")

20    def get(self, key):
21        index = self._hash_function(key)
22        bucket = self.table[index]

```



```

19] 0s
    # Search for the key in the bucket
    for k, v in bucket:
        if k == key:
            print(f"Found key '{key}' with value '{v}' at index {index}.")
            return v

    print(f"Key '{key}' not found.")
    return None

print("HashTable class defined successfully with __init__, _hash_function, insert, and get methods.")

... HashTable class defined successfully with __init__, _hash_function, insert, and get methods.
print(
    "\n--- Testing HashTable get method ---"
)
hash_table = HashTable(size=5)

# Insert some key-value pairs
hash_table.insert("apple", 10)
hash_table.insert("banana", 20)
hash_table.insert("cherry", 30)
hash_table.insert("date", 40)

print(
    "\n--- Retrieving values ---"
)
# Retrieve existing keys
value = hash_table.get("apple")
print(f"Retrieved 'apple': {value}")

value = hash_table.get("cherry")
print(f"Retrieved 'cherry': {value}")

# Retrieve a non-existent key
value = hash_table.get("grape")
print(f"Retrieved 'grape': {value}")

# Test with an updated key (from previous insert test)
hash_table.insert("apple", 15)
value = hash_table.get("apple")

print(f"Retrieved updated 'apple': {value}")

print(
    "\n--- HashTable get testing complete ---"
)

...
--- Testing HashTable get method ---
HashTable initialized with size: 5.
Inserted key 'apple' with value '10' at index 4.
Inserted key 'banana' with value '20' at index 1.
Inserted key 'cherry' with value '30' at index 0.
Inserted key 'date' with value '40' at index 2.

--- Retrieving values ---
Found key 'apple' with value '10' at index 4.
Retrieved 'apple': 10
Found key 'cherry' with value '30' at index 0.
Retrieved 'cherry': 30
Key 'grape' not found.
Retrieved 'grape': None
Updated key 'apple' at index 4.
Found key 'apple' with value '15' at index 4.
Retrieved updated 'apple': 15
--- HashTable get testing complete ---

```


palette (Ctrl+Shift+P)

Insert Operations ---

```
# Insert unique key-value pairs
test_hash_table.insert("apple", 10)
test_hash_table.insert("banana", 20)
test_hash_table.insert("cherry", 30)
test_hash_table.insert("date", 40)
test_hash_table.insert("grape", 50) # Expected collision based on typical hash() behavior and size=5
test_hash_table.insert("elderberry", 60) # Another insertion
```

```
# Determine some keys that might collide for size 5
# We'll calculate hash values for some common fruits to find collisions.
# Let's say: hash('fig') % 5 == hash('apple') % 5, or similar.
# For demonstration, we'll just add more and trust default hash() to create some collisions.
test_hash_table.insert("fig", 70) # Potentially causes another collision
```

```
# Insert a key that already exists to test update functionality
test_hash_table.insert("apple", 15) # Update 'apple's value
test_hash_table.insert("banana", 25) # Update 'banana's value
```

```
# --- 3. Display State after insertions and updates ---
print("\n--- Current state of the HashTable after initial insertions and updates ---")
print(test_hash_table)
```

```
# --- 4. Get Operations ---
print("\n--- Testing Get Operations ---")
```

```
# Retrieve existing keys
```

▶ # Retrieve existing keys

```
print(f"Retrieving 'apple': {test_hash_table.get('apple')}") # Should be 15
print(f"Retrieving 'cherry': {test_hash_table.get('cherry')}") # Should be 30
print(f"Retrieving 'fig': {test_hash_table.get('fig')}") # Should be 70
```

```
# Attempt to get a non-existent key
print(f"Retrieving 'kiwi': {test_hash_table.get('kiwi')}") # Should be None
```

```
# --- 5. Delete Operations ---
print("\n--- Testing Delete Operations ---")
```

```
# Delete a key that might be at the 'head' of a chain (first in a bucket)
# This depends on the hash function, for now, let's pick a key we know exists.
print(f"Deleting 'banana': {test_hash_table.delete('banana')}")
```

```
# Delete a key that might be in the 'middle' or 'end' of a chain.
# This is hard to guarantee without knowing the exact hash collisions upfront.
# Let's delete another existing key.
print(f"Deleting 'date': {test_hash_table.delete('date')}")
```

```
# Attempt to delete a non-existent key
print(f"Deleting 'mango': {test_hash_table.delete('mango')}")
```

```
# Delete a key that might leave a bucket empty
print(f"Deleting 'cherry': {test_hash_table.delete('cherry')}")
```

```

# --- 6. Display Final State ---
print("\n--- Final state of the HashTable after deletions ---")
print(test_hash_table)
print("--- HashTable testing complete ---")

...
--- Thorough Testing of HashTable Implementation ---
HashTable initialized with size: 5.
Inserted key 'apple' with value '10' at index 4.
Inserted key 'banana' with value '20' at index 1.
Inserted key 'cherry' with value '30' at index 0.
Inserted key 'date' with value '40' at index 2.
Inserted key 'grape' with value '50' at index 3.
Inserted key 'elderberry' with value '60' at index 0.
Inserted key 'fig' with value '70' at index 3.
Updated key 'apple' at index 4.
Updated key 'banana' at index 1.

--- Current state of the HashTable after initial insertions and updates ---
Bucket 0: [(cherry: 30), (elderberry: 60)]
Bucket 1: [(banana: 25)]
Bucket 2: [(date: 40)]
Bucket 3: [(grape: 50), (fig: 70)]
Bucket 4: [(apple: 15)]

--- Testing Get Operations ---
Found key 'apple' with value '15' at index 4.
Retrieving 'apple': 15
Found key 'cherry' with value '30' at index 0.
Retrieving 'cherry': 30

Found key 'fig' with value '70' at index 3.
Retrieving 'fig': 70
.. Key 'kiwi' not found.
Retrieving 'kiwi': None

--- Testing Delete Operations ---
Deleted key 'banana' from index 1.
Deleting 'banana': True
Deleted key 'date' from index 2.
Deleting 'date': True
Key 'mango' not found for deletion.
Deleting 'mango': False
Deleted key 'cherry' from index 0.
Deleting 'cherry': True

--- Final state of the HashTable after deletions ---
Bucket 0: [(elderberry: 60)]
Bucket 1: []
Bucket 2: []
Bucket 3: [(grape: 50), (fig: 70)]
Bucket 4: [(apple: 15)]
--- HashTable testing complete ---

```

Explanation : The current hash table uses Python's built-in hash() function, which is generally efficient but can be non-deterministic across different Python interpreter runs or versions for certain custom objects. For production systems or specific use cases, consider implementing a custom hash function that provides more predictable and consistent behavior, especially for complex object keys

Task 5: Real-Time Application Challenge

Scenario

Design a **Campus Resource Management System** with the following features:

- Student Attendance Tracking
- Event Registration System
- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue

Student Tasks

1. Choose the most appropriate data structure for each feature.
2. Justify your choice in **2–3 sentences**.
3. Implement **one selected feature** using AI-assisted code generation.

Expected Outcome

- Mapping table: Feature → Data Structure → Justification
- One fully working Python implementation

Prompt :

Design a Campus Resource Management System with features like student attendance tracking, event registration, library book borrowing, bus scheduling, and cafeteria order queues. For each feature, choose the most suitable data structure and justify your choice in 2–3 sentences. Then, implement one selected feature in Python using AI-assisted code generation to demonstrate functionality.

Code :

```
[26]
✓ 0s class Order:
      def __init__(self, order_id, item):
          self.order_id = order_id
          self.item = item

      def __str__(self):
          return f"Order ID: {self.order_id}, Item: {self.item}"

      print("Order class defined successfully.")

... Order class defined successfully.
```

```

class CafeteriaQueue:
    def __init__(self):
        self.queue = []
        print("CafeteriaQueue initialized.")

    def enqueue(self, order):
        self.queue.append(order)
        print(f"Enqueued order: {order.order_id} - {order.item}")

    def dequeue(self):
        if self.is_empty():
            print("Queue is empty. No orders to dequeue.")
            return None
        order = self.queue.pop(0)
        print(f"Dequeued order: {order.order_id} - {order.item}")
        return order

    def is_empty(self):
        return len(self.queue) == 0

    def __len__(self):
        return len(self.queue)

    def __str__(self):
        if self.is_empty():
            return "Cafeteria Queue: Empty"
        orders_str = "\n".join(str(order) for order in self.queue)

```

Toqale Gemini

```

        return f"Cafeteria Queue (Front to Back):\n{orders_str}"

```

```

print("CafeteriaQueue class defined successfully.")

```

CafeteriaQueue class defined successfully.

Reasoning: Now that the `CafeteriaQueue` class is defined, I will demonstrate its functionality by creating an instance, enqueueing several `Order` objects, checking the queue's state, dequeuing orders, and verifying the `is_empty` and `__str__` methods. This will confirm that the FIFO queue for cafeteria orders behaves as expected.

```

print("\n--- Testing CafeteriaQueue ---")
cafeteria_queue = CafeteriaQueue()

# Create some order objects
order1 = Order("C-001", "Burger and Fries")
order2 = Order("C-002", "Salad and Water")
order3 = Order("C-003", "Pizza Slice and Soda")

# Enqueue orders
cafeteria_queue.enqueue(order1)
cafeteria_queue.enqueue(order2)
cafeteria_queue.enqueue(order3)

print("\nCurrent queue state:")
print(cafeteria_queue)

```

```

print("\nDequeuing first order:")
processed_order = cafeteria_queue.dequeue()
if processed_order:
    print(f"Processed: {processed_order}")

print("\nCurrent queue state:")
print(cafeteria_queue)

print("\nDequeuing second order:")
processed_order = cafeteria_queue.dequeue()
if processed_order:
    print(f"Processed: {processed_order}")

print("\nCurrent queue state:")
print(cafeteria_queue)

print("\nDequeuing third order:")
processed_order = cafeteria_queue.dequeue()
if processed_order:
    print(f"Processed: {processed_order}")

print("\nFinal queue state:")
print(cafeteria_queue)
print(f"Is queue empty? {cafeteria_queue.is_empty()}")

print("\nAttempting to dequeue from empty queue:")
cafeteria_queue.dequeue()

```

```

print("--- CafeteriaQueue testing complete ---")

```

```

...

```

```

--- Testing CafeteriaQueue ---
CafeteriaQueue initialized.
Enqueued order: C-001 - Burger and Fries
Enqueued order: C-002 - Salad and Water
Enqueued order: C-003 - Pizza Slice and Soda

Current queue state:
Cafeteria Queue (Front to Back):
Order ID: C-001, Item: Burger and Fries
Order ID: C-002, Item: Salad and Water
Order ID: C-003, Item: Pizza Slice and Soda

Dequeuing first order:
Dequeued order: C-001 - Burger and Fries
Processed: Order ID: C-001, Item: Burger and Fries

Current queue state:
Cafeteria Queue (Front to Back):
Order ID: C-002, Item: Salad and Water
Order ID: C-003, Item: Pizza Slice and Soda

Dequeuing second order:
Dequeued order: C-002 - Salad and Water
Processed: Order ID: C-002, Item: Salad and Water

Current queue state:
Cafeteria Queue (Front to Back):
Order ID: C-003, Item: Pizza Slice and Soda

```

```

Dequeuing third order:
Dequeued order: C-003 - Pizza Slice and Soda
Processed: Order ID: C-003, Item: Pizza Slice and Soda

```

```

Final queue state:
Cafeteria Queue: Empty
Is queue empty? True

```

```

Attempting to dequeue from empty queue:
Queue is empty. No orders to dequeue.
--- CafeteriaQueue testing complete ---

```

- Explanation : The Order class was successfully defined to hold order id and item details. The CafeteriaQueue class was correctly

	implemented with enqueue, dequeue, is_empty, __len__, and __str__ methods. Testing demonstrated the queue's proper FIFO (First-In, First-Out) behavior, with orders processed in the exact sequence they were added, and correct handling for dequeuing from an empty queue.	
--	--	--